SAVEETHA SCHOOL OF ENGINEERING
CSA04 – OPERATING SYSTEMS
ASSIGNMENT 1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Name: _____

Register Number: _____

Department: Computer Science and Engineering

Course Code: CSA04

Total Marks: 50

Submission Date: 02 November 2026

GitHub Repository Link: _____

Submission and execution note

This report follows the supplied assignment and its twelve tasks. Replace student and GitHub placeholders. Expected outputs are distinguished from machine-dependent measurements. Run the supplied programs on your own Linux environment and retain your own terminal evidence.

Contents

TASK 1 – Requirements of Each Computing Environment (4 marks)

TASK 2 – Types of Operating Systems (3 marks)

TASK 3 – Operating-System Structures (4 marks)

TASK 4 – Operating-System Services (4 marks)

TASK 5 – System Calls (4 marks)

TASK 6 – Cost of System Calls (4 marks)

TASK 7 – Process Control System Calls (4 marks)

TASK 8 – CPU States and Protection (3 marks)

TASK 9 – Memory Hierarchy and I/O Systems (4 marks)

TASK 10 – Measure and Compare (4 marks)

TASK 11 – Recommended Hospital OS Plan (4 marks)

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

TASK 1 – Requirements of Each Computing Environment (4 marks)

The hospital has six distinct computing environments. Requirements are ranked by patient-safety impact first, then availability, security, response time, throughput and energy efficiency.

Environment	Ranked requirements (1 = highest)	Justification
Bedside monitors (500)	1. Deterministic response ≤10 ms; 2. reliability; 3. sensor I/O accuracy; 4. integrity/security; 5. low memory/power	Missing an alarm deadline can harm a patient. A hard real-time design must bound worst-case response, not merely provide a good average.
Nurses' tablets (120)	1. security/privacy; 2. battery life for a shift; 3. wireless availability; 4. responsive UI; 5. manageable storage	Tablets are mobile and frequently disconnected. They need encryption, authentication, power management and safe offline handling.
Doctors' workstations (80)	1. security; 2. usability/responsiveness; 3. reliable access; 4. multitasking; 5. peripheral support	Clinical users need a responsive desktop UI, controlled access to records and support for approved imaging/clinical applications.
Patient-record database	1. availability and durability; 2. confidentiality/integrity; 3. transaction correctness; 4. throughput; 5. recoverability	A database outage or corrupted record affects care. Redundancy, backups, access control and transaction logging are essential.
MRI imaging server	1. throughput; 2. memory bandwidth/capacity; 3. availability; 4. parallel processing; 5. data integrity	MRI reconstruction is compute- and memory-intensive; multiple processors and high-speed storage/network I/O are useful.
Partner-hospital link	1. confidentiality/integrity; 2. availability; 3. interoperability; 4. throughput; 5. auditability	Shared records require authenticated encrypted communication, authorization, audit logs and resilient network connectivity.

Conflicts and trade-offs:

• Strict real-time scheduling reserves predictable CPU capacity, potentially reducing general-purpose throughput.

• Strong encryption and audit logging consume CPU and storage, but cannot be removed from patient-record systems.

• More background jobs and users increase throughput demand but can introduce scheduling delays.

• Tablets need rich clinical applications but limited battery and storage; background activity must be controlled.

• Replication improves availability but introduces consistency, network and operational complexity.

OS history applied:

• Batch systems suit scheduled backups and bulk image processing, not interactive alarms.

• Time-sharing supports many doctors and nurses using shared servers.

• Personal-computer OSs suit desktop workstations.

• Mobile OS principles (power management, wireless roaming, sandboxing) suit tablets.

• Real-time OS principles suit bedside monitors because deadlines matter.

• Distributed-system principles support three hospitals sharing records.

Single general-purpose OS alternative:

A single general-purpose OS simplifies administration and staff training, but ordinary time-sharing schedulers generally do not promise a hard 10 ms worst-case response. Under overload, contention, interrupts or driver delays, scheduling latency may exceed the deadline. It also has a larger memory/storage footprint than a small embedded RTOS. A common OS image can reduce administration variation, but a single configuration cannot simultaneously optimize deterministic response, tablet battery life, desktop usability and clustered database availability. A general-purpose OS with real-time extensions and strict configuration could reduce the gap, but must be validated against worst-case execution and hardware timing. Therefore, use purpose-fit OS profiles rather than one identical OS everywhere.

TASK 2 – Types of Operating Systems (3 marks)

Hospital component	Recommended OS type	Reason and rejected alternatives
Bedside monitors	Embedded hard real-time OS (e.g., QNX or suitably configured RTOS)	Bounded interrupt and task latency supports the 10 ms alarm deadline. A general-purpose desktop OS does not inherently guarantee it.
Nurses' tablets	Handheld/mobile OS	Integrated power management, touch UI, wireless connectivity and application sandboxing. A server OS is unnecessarily large.

Hospital component	Recommended OS type	Reason and rejected alternatives
Doctors' workstations	Desktop OS	Supports interactive GUI, clinical applications, peripherals and multitasking. A handheld OS would constrain workstation software.
Patient database	Clustered server OS deployment	Two or more nodes with shared/replicated storage, health checks and automatic fail-over reduce a single point of failure.
MRI imaging	Multiprocessor server OS	Uses multiple CPUs/cores and large memory bandwidth for parallel reconstruction.
Partner hospitals	Distributed-system architecture over secure network links	Each hospital retains local services while exchanging authorized records through interoperable APIs and replication.

OS type comparison:

• Mainframe: high-volume centralized transaction processing; possible for very large centralized workloads, but not necessary as the only hospital platform.

• Desktop: interactive end-user computing.

• Multiprocessor: one computer uses multiple CPUs/cores; suitable for MRI processing.

• Distributed: multiple networked computers cooperate; suitable for multi-hospital records.

• Clustered: coordinated servers provide fail-over and service continuity; suitable for the database.

• Hard real-time: missing a deadline is unacceptable; bedside alarm path.

• Soft real-time: occasional deadline misses degrade quality but are not catastrophic; some imaging/display workloads may fit.

• Handheld: power-aware mobile computing; tablets.

SMP versus AMP for MRI:

Symmetric multiprocessing (SMP) lets processors share memory and the OS scheduler balance work dynamically. It is generally the practical choice for a general-purpose imaging server because workloads can be parallelized and load-balanced. Asymmetric multiprocessing (AMP) assigns processors fixed roles; it can be useful for specialized pipelines or isolated control tasks, but can create imbalance and requires explicit partitioning. Choose SMP for the main reconstruction server, with dedicated accelerators where justified.

Cluster versus one large multiprocessor:

A single large server may have lower initial complexity and excellent local interconnect performance, but remains a single failure domain: motherboard, power, OS crash or maintenance can stop the database. A cluster can fail over to another node within seconds if correctly configured and tested. Clustering costs more in hardware, networking, storage/replication and administration. Throughput can increase with processors, but not linearly because of synchronization, memory bandwidth and serial portions of the workload (Amdahl's law). Memory capacity is limited by the single machine in a scale-up design; a cluster can add aggregate memory, although application data partitioning and distributed caching are required. For records that must remain available, a cluster is preferred.

TASK 3 – Operating-System Structures (4 marks)

Structure	Design	Reliability	Performance	Updating/maintenance
Monolithic/simple	Most OS services and drivers run in kernel space	A faulty driver can crash or corrupt the whole kernel	Usually low internal communication overhead	Changes can affect the whole kernel
Layered	OS divided into layers with defined interfaces	Clear boundaries aid reasoning and testing	Layer crossings can add overhead	Easier to isolate and replace layers
Microkernel	Minimal kernel; services/drivers run as user-space servers communicating by messages	Strong fault isolation; failed service may be restarted	Message passing/context switches can add overhead	Services can be updated independently
Modular	Core kernel plus loadable modules	Flexible, but a faulty kernel module can still crash kernel	Near-monolithic performance for many operations	Modules can be loaded/unloaded without rebuilding the whole kernel
Hybrid	Combines microkernel ideas with substantial kernel-space services	Depends on implementation; some isolation with practical performance	Designed to reduce microkernel overhead	Componentized but still complex

Recommended structures:

1. Bedside monitors: a small microkernel-oriented RTOS is preferred where certified drivers and services can be isolated. Keep only the alarm path, sensor I/O, timer, scheduling, watchdog and essential logging. This limits the impact of noncritical service failure. Validate actual worst-case latency on the target hardware.

2. Database server: a mature modular monolithic kernel such as Linux is appropriate because it provides high-throughput networking, storage drivers, filesystems, observability and mature server tooling. Use a minimal, tested module set and controlled updates.

Platform examples:

• Linux: monolithic kernel with loadable modules (often described as modular monolithic).

• Windows: hybrid kernel architecture.

• Android: Linux-kernel-based stack with Android framework and HAL layers; kernel remains modular monolithic.

• FreeRTOS: small real-time kernel, commonly used in embedded systems; it is not a full desktop OS.

• QNX: microkernel-based real-time OS.

Monolithic alternative for bedside monitors:

A monolithic design can make a service call with fewer internal message-passing steps and often lower overhead. A microkernel may require a user-to-kernel transition plus IPC to a server and possibly a reply, adding latency. However, the microkernel can keep drivers/services outside the kernel and reduce fault propagation. A monolithic kernel may occupy more kernel memory because more services/drivers are resident, though actual memory depends on configuration. A faulty in-kernel sensor driver can crash or corrupt the whole monitor. The correct decision is not based on average call speed alone: the 10 ms worst-case deadline and fault containment make a small, validated RTOS/microkernel design preferable.

TASK 4 – Operating-System Services (4 marks)

OS service	Monitors	Tablets	Workstations	Database	MRI server
User interface	Minimal alarm/status UI	Touch UI	Full GUI	Admin/monitoring tools	Admin/processing UI
Program execution	Fixed validated tasks	Clinical apps	Clinical apps	DB engine/services	Reconstruction software
I/O operations	Sensor, alarm, display	Touch, Wi-Fi, camera	Keyboard, display, network	Storage/network	Scanner/network/storage
File system	Small configuration/log store	Encrypted local cache	User/app files	Journals, data, backups	Large image datasets
Communication	Sensor buses, alarm network	Wi-Fi/VPN/API	LAN/API	Replication/client protocol	High-speed data transfer
Error detection	Watchdog, sensor validation	App/system health	System logs	ECC, checksums, DB recovery	ECC, job validation
Resource allocation	Fixed priorities/reservations	CPU and battery management	Multitasking	CPU, memory, I/O scheduling	Parallel CPU/GPU/memory
Accounting	Minimal event logs	Usage/security logs	Audit logs	Detailed transaction/audit logs	Job/resource logs
Protection/security	Secure boot, signed updates, restricted interfaces	Encryption, MFA, sandbox	RBAC, endpoint protection	Least privilege, encryption, audit	Access control and network isolation

Services to minimize/remove from bedside monitors:

No general-purpose desktop shell, app store, print spooler, multimedia services, broad user account system, unnecessary network listeners or nonessential background daemons. Retain alarm scheduling, sensor drivers, watchdog, timekeeping, authenticated communication, secure boot, integrity checks and essential audit/event logs. Removing nonessential services reduces attack surface and resource contention.

Database essentials:

Reliable filesystem/storage stack, process and memory management, networking, synchronization, resource allocation, access control, encryption support, audit logging, error detection, backups, transaction recovery and monitoring.

Protecting one patient's records:

Use authenticated user identities, role-based access control (RBAC), least privilege, per-session authorization, database views/row-level controls where supported, process isolation, filesystem permissions, encryption in transit and at rest, audit trails, secure key management and session timeouts. A program must not gain access merely because it runs on the same server. Database authorization must be checked for each query/operation.

Full desktop OS on tablets alternative:

A full desktop OS may offer broad software compatibility, but extra services increase RAM, storage, background CPU use and update burden. More background work can increase startup time and reduce battery life. A reduced mobile OS generally offers better integrated power management and app sandboxing. Exact overhead depends on the chosen OS image and enabled services; measure on actual tablet hardware rather than inventing a fixed figure.

TASK 5 – System Calls (4 marks)

System calls are the controlled interface through which a user program requests privileged OS services.

Categories and examples:

• Process control: fork(), execve(), wait(), exit(), kill().

• File management: open(), read(), write(), close(), stat(), unlink().

• Device management: ioctl(), read(), write(); device-specific operations are mediated by drivers.

• Information maintenance: getpid(), uname(), clock_gettime(), gettimeofday().

• Communication: pipe(), socket(), send(), recv(), shmget() (API availability varies).

• Protection: chmod(), chown(), setuid(), access(); modern systems also use capabilities and security modules.

What happens during read(fd, buffer, count):

1. Application calls the C library wrapper read().

2. The wrapper places the system-call number and arguments in the architecture-defined registers/ABI locations.

3. A syscall/trap instruction transfers control to the kernel and changes CPU privilege from user mode to kernel mode.

4. Kernel validates the descriptor, permissions and user buffer; it locates the file/device operation.

5. VFS/filesystem or device driver performs the operation; the process may block if data is not ready.

6. Kernel returns a byte count or -1 with errno set; CPU returns to user mode.

7. The library wrapper returns to the application.

Parameter passing:

Most modern Linux architectures pass syscall number and arguments in registers. Some interfaces can pass a pointer to a structure/block in memory. The stack is used by normal calling conventions and may hold data, but the kernel ABI defines how syscall arguments are passed; it is not universally “the stack”.

strace:

`strace -o trace.txt ./program` records system calls and signals.

`strace -c ./program` summarizes syscall counts and time. Output includes calls made by the program and runtime/library startup, so isolate the relevant calls when comparing.

Buffered stdio alternative:

fopen(), fread() and fwrite() are C library functions, not direct system calls. They maintain a user-space buffer. For a file of n bytes and a library buffer of about 4096 bytes, data transfers can require roughly ceil(n/4096) read/write syscalls rather than one syscall per tiny application read/write of b bytes (roughly ceil(n/b), plus EOF/metadata effects). Buffering consumes about the buffer size per stream and may add a copy between the library buffer and the application buffer. It usually improves throughput for small operations; direct system calls offer more control.

TASK 6 – Cost of System Calls (4 marks)

A system call is more expensive than an ordinary function call because it crosses the user/kernel boundary, saves/restores state, validates arguments and permissions, may interact with drivers/filesystems, and may block or trigger scheduling. The exact cost depends on CPU, kernel, filesystem, cache state and workload.

For the supplied 10,000-byte test file, the program counts every read(), including the final read returning 0 (EOF):

• buffer 1: 10,000 data reads + 1 EOF read = 10,001.

• buffer 512: ceil(10,000/512) = 20 data reads + 1 EOF read = 21.

• buffer 4096: ceil(10,000/4096) = 3 data reads + 1 EOF read = 4.

```
Expected output:
buffer    1 bytes: 10001 read() calls
```

```
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

The write loop must handle partial writes and EINTR. A single write() is not guaranteed to write every requested byte. The provided implementation below does so.

Recommendation:

For large sequential backups, use a moderate/large buffer such as 64 KiB or 256 KiB after benchmarking. The assignment's tested maximum is 4096 bytes, which reduces calls dramatically versus 1 byte. Do not claim 4096 is universally optimal; benchmark on the actual storage and workload. The fixed source array is 8192 bytes, so bufsize must not exceed 8192.

Complexity:

For n bytes copied, time is O(n) because each byte is read and written once. Auxiliary space is O(b), where b is buffer size. Increasing b reduces syscall count approximately to O(n/b), but does not change O(n) byte-processing complexity.

mmap alternative:

mmap() maps file pages into the process address space; the program accesses mapped memory rather than issuing repeated read() calls. It can reduce explicit read/write calls and avoid an extra user-space copy in some workflows, but page faults still cause kernel work, dirty pages need writeback, mapping/unmapping and synchronization have costs, and mapping very large files consumes virtual address space (not necessarily equal physical RAM). For huge files, map in windows/chunks or retain buffered I/O for simpler bounded memory use.

TASK 7 – Process Control System Calls (4 marks)

fork() creates a child process with a logically separate address space (implemented efficiently using copy-on-write on Linux). exec() replaces the current process image with a new program; it does not create another process. wait()/waitpid() collects a terminated child’s status and prevents a zombie from remaining. exit() terminates a process and runs normal user-space exit handlers; _exit() terminates without flushing C stdio buffers or running atexit handlers.

Shell sequence:

1. Shell reads and parses a command.

2. Shell calls fork() (or a spawn API).

3. Child calls exec() to load the requested program.

4. Parent calls waitpid() for a foreground command.

5. Kernel records termination status; parent obtains it with WIFEXITED/WEXITSTATUS or WIFSIGNALED.

6. Shell displays prompt/continues.

Zombie: a child has terminated but its parent has not yet collected its status. The kernel retains a small process-table entry. Parent should wait(). Orphan: parent terminates while child is running; the child is re-parented to an init/subreaper process, which eventually reaps it.

Why fork()+exec():

It separates process creation from program replacement and lets the child configure file descriptors, pipes, environment and redirections between the two calls. posix_spawn() packages common spawn actions more efficiently in some environments.

Expected supplied-program output:

```
shell: running job 1
```

42

job 1 exit status 0

```
shell: running job 2
```

job 2 exit status 3

stdout flushing:

Before fork(), stdout may be line-buffered on a terminal but fully buffered when redirected to a file. The child inherits a copy of the user-space stdio buffer. Flushing before fork prevents both parent and child from later flushing the same buffered text. In the second experiment, removing the second fflush and using exit(3) causes the child to flush its inherited buffer at exit, while the parent later flushes its own copy; therefore “shell: running job 2” can appear twice in redirected output. _exit(3) does not flush stdio.

Alternative:

posix_spawn() can avoid a general fork-then-exec sequence and may be efficient for launching many programs. Threads share one address space and resources, so creation can be lighter, but they are not equivalent to isolated commands: a faulty or malicious thread can corrupt shared memory. For independent shell commands, processes provide stronger isolation. For 1,000 launches, measure on the target OS; exact times depend on machine load and implementation. Complexity of creating k children/threads is O(k) operations; per-process memory overhead is generally higher than per-thread overhead, while fork uses copy-on-write rather than eagerly copying all pages.

TASK 8 – CPU States and Protection (3 marks)

User mode restricts application instructions and memory access. Kernel mode permits privileged operations such as configuring devices, page tables and interrupt controls. A hardware mode bit/privilege level distinguishes execution privilege. A user program requests privileged work through a system call, exception or interrupt.

Interrupts are asynchronous events from hardware (for example, a device signaling data ready). A trap/exception is a synchronous event caused by the current instruction or an explicit syscall. A timer interrupt lets the scheduler regain control, pre-empt a running task and enforce time sharing/priorities.

If any monitor program could disable interrupts, a faulty or malicious task could prevent timely sensor/alarm handling and starve other tasks. Therefore, interrupt control must remain privileged and only trusted kernel/driver code may manage it.

Memory protection:

The MMU and page tables map each process's virtual address space and enforce read/write/execute permissions. User-mode code cannot access kernel pages or another process's pages unless the OS deliberately shares memory. The kernel validates pointers and permissions at system-call boundaries.

Polling alternative:

If a sensor is checked every P milliseconds, the worst-case detection wait is approximately P plus processing/scheduling delay. To guarantee a 10 ms alarm deadline, P and all other worst-case delays must fit within 10 ms. Interrupt-driven notification can respond closer to interrupt latency and avoids continuously spending CPU cycles checking unchanged data. Polling may be simpler and can be predictable with a carefully chosen interval, but frequent polling wastes CPU and battery; infrequent polling increases detection delay. For safety-critical monitors, use validated interrupt-driven I/O with bounded latency, while retaining watchdog/timeouts as a fallback.

TASK 9 – Memory Hierarchy and I/O Systems (4 marks)

Typical hierarchy (approximate values; hardware varies):

Level	Typical latency	Relative capacity	Hospital use
CPU registers	<1 ns	Bytes–KB per core	Current instructions and operands
L1/L2/L3 cache	~1–20 ns	KB–tens of MB	Hot database indexes and processing data
DRAM	~50–100 ns	GBs–TBs	Active applications, DB cache, MRI working sets
NVMe SSD	~10–100 µs	100s GB–TBs	Patient records, database logs, recent MRI images
HDD	~5–15 ms seek (plus transfer)	TBs	Large image archive and backups
Network/object storage	~sub-ms to many ms or more	Scalable	Replicated records, off-site archive, shared hospital data

Data placement:

• Monitor readings: keep the newest values in local RAM for immediate alarm processing; persist critical events to durable local storage and securely forward them to central records.

• Patient records: durable database storage on redundant SSD-backed storage with transaction logs, backups and replicated copies.

• MRI images: fast local NVMe/DRAM scratch space during reconstruction, then durable PACS/archive storage; use network storage for sharing and long-term retention.

• Caches: keep frequently used patient indexes/metadata in memory with authorization checks and invalidation/consistency rules. Caching reduces repeated storage/network latency but stale clinical data must not be presented as current without validation.

I/O methods:

• Programmed I/O: CPU repeatedly checks device status and moves data; simple but wastes CPU.

• Interrupt-driven I/O: device interrupts CPU when service is needed; suitable for event-driven low-volume device interactions.

• DMA: device/controller transfers blocks directly between device and RAM with limited CPU involvement; suitable for large MRI image transfers.

500 MB interrupt-per-word alternative:

Assume decimal 500 MB = 500,000,000 bytes and 4-byte words. Interrupts = 125,000,000. If “500 MiB” is intended, 524,288,000 / 4 = 131,072,000 interrupts. This is an enormous interrupt rate and would consume substantial CPU time. With DMA, a small number of interrupts can signal completion per block/batch, so CPU overhead is much lower. DMA needs appropriately sized buffers/descriptors and cache-coherency handling; interrupt-driven transfer may use smaller buffers but requires servicing every word/event. The assumption about word size must be stated because the question does not specify it.

TASK 10 – Measure and Compare (4 marks)

Required commands (run from the task directory on Linux):

1. Compile: `gcc -std=c11 -Wall -Wextra -O2 file_copy.c -o file_copy`

2. Run: `./file_copy`

3. Trace summary: `strace -c ./file_copy`

4. Timing: `/usr/bin/time -v ./file_copy` (or `time ./file_copy`)

5. Process/thread benchmark: `gcc -std=c11 -Wall -Wextra -O2 process_thread_bench.c -o process_thread_bench -pthread && ./process_thread_bench`

Interpretation:

• Increasing copy buffer size should reduce read()/write() syscall counts approximately in proportion to n/b. The actual strace summary also includes startup, file-open/close, write and runtime calls.

• `time` reports elapsed (wall-clock), user CPU and system CPU time. System time generally reflects kernel work; user time reflects application execution. Tiny tests may round to 0.00 seconds, so repeat runs or increase workload.

• Process creation normally costs more memory and kernel bookkeeping than thread creation. Threads share the process address space and most resources; each still needs a stack and thread-control data. fork() uses copy-on-write on Linux, so memory pages are not eagerly duplicated unless modified.

• Measurements are machine- and load-dependent. Record actual results, OS/kernel/GCC versions and test conditions. Do not copy another machine’s timings as personal measurements.

Expected qualitative result:

Experiment	Expected trend
Buffer 1 → 512 → 4096 bytes	Read calls fall 10001 → 21 → 4 for the provided 10,000-byte input
Larger buffers	Fewer syscall transitions, usually better sequential-copy throughput until diminishing returns
1,000 threads vs 1,000 processes	Threads usually start with lower overhead; both take measurable time and are subject to system limits
user/system time	System time is more affected by syscall/process-management overhead; user time includes loop and application work

The included benchmark prints measured elapsed time on the machine where it is executed. Its values are not predetermined expected constants.

TASK 11 – Recommended Hospital OS Plan (4 marks)

Recommended deployment:

Environment	OS type/structure	Services and protection	I/O and availability
Bedside monitors	Embedded hard RTOS; small microkernel-oriented design	Fixed-priority tasks, secure boot, signed updates, watchdog, minimal services	Interrupt-driven sensor I/O; bounded alarm path; local fail-safe alarm
Nurses’ tablets	Handheld/mobile OS	Device encryption, MFA, sandboxing, remote management, offline cache controls	Wi-Fi with secure VPN/API; battery-aware scheduling
Doctors’ workstations	Desktop OS	RBAC, endpoint protection, disk encryption, audit	LAN/Wi-Fi, standard peripherals
Patient database	Clustered Linux server OS; modular monolithic kernel	DB transactions, least privilege, encryption, audit, backups, replication	Redundant network/storage, health checks and automatic fail-over

Environment	OS type/structure	Services and protection	I/O and availability
MRI server	SMP multiprocessor Linux server	Parallel scheduling, resource monitoring, validated drivers	DMA-capable high-speed I/O, NVMe and high-speed network
Partner hospitals	Distributed services	Mutual authentication, authorization, encryption, audit and data governance	Secure APIs, replication and retry/idempotency

Risks/limitations:

• RTOS and monitor hardware require formal timing validation, safety testing and controlled change management.

• Clusters need quorum, split-brain protection, tested backups and regular fail-over drills.

• Distributed records can have network partitions and replication delay; clinical workflows need clear source-of-truth and conflict handling.

• Mobile devices can be lost or disconnected; remote wipe and offline-data expiry are needed.

• Vendor compatibility, patching and staff training add operational cost.

Three-hospital suitability:

The design scales by retaining local real-time monitoring and local clinical operations at each hospital while using secure distributed services for record exchange. A central/cloud service can provide shared indexes and replicated records, but each site should have a local cache/continuity plan for network outages.

Two large virtualized servers alternative:

Virtualization can consolidate workloads and improve deployment flexibility, but placing hard real-time bedside monitoring on shared virtual hosts introduces hypervisor scheduling, device virtualization and noisy-neighbor latency. Memory and CPU overhead are incurred by the hypervisor and guest OSs; overcommit can cause unpredictable delays. Two servers also create correlated failure risk unless storage, networking and fail-over are carefully designed. Keep bedside monitor control on dedicated validated embedded hardware; virtualize suitable database/app services with reservations, redundancy and tested recovery.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

Decision: MODIFY the original design, rather than retain it unchanged or redesign everything.

A. Impact analysis (2 marks)

Home monitoring introduces untrusted networks, variable latency, intermittent connectivity, device diversity and a need for secure device identity. The cloud records service introduces multi-tenant isolation, shared identity, API security, data residency/governance, encryption, audit, backup and disaster recovery. The original local monitor RTOS remains relevant, but communication, security and distributed services must expand.

B. Evaluation of original design (2 marks)

The hospital's local real-time design remains appropriate for alarms generated at the bedside. It cannot assume internet delivery will meet the same 10 ms deadline: home Wi-Fi, broadband, cellular networks and cloud queues have variable latency and outages. Therefore, safety-critical local alarms must be generated on the device itself; remote transmission is supplementary. The original clustered database can continue local operation, but shared cloud records require secure replication, conflict resolution, availability design and strong identity/access controls.

C. Adaptation (2 marks)

• Home device: embedded RTOS or safety-appropriate embedded OS; local threshold detection, audible/visual alarm where appropriate, secure boot, signed firmware, encrypted storage and device certificates.

• Gateway/mobile app: mobile OS with authenticated pairing, encrypted communication, buffering during outages and user consent.

• Connectivity: TLS-protected authenticated APIs or message broker; device identity, certificate rotation, replay protection, rate limiting and monitoring.

• Cloud records: distributed, highly available service with encrypted databases, tenant/site isolation, role-based access, immutable audit logs, backups and disaster recovery.

• Hospital edge: local cache and store-and-forward gateway so care workflows continue during cloud/network outages.

• Interoperability: use agreed healthcare data formats/APIs and validate data provenance, timestamps, units and patient matching.

D. Evidence-based defence (2 marks)

Modify is the balanced choice because it preserves the validated local real-time alarm path while extending secure communication and distributed storage. A complete redesign would add cost and risk without removing the need for local deterministic monitoring. Retaining the design unchanged would fail to address remote-device identity, internet outages, cloud access control and multi-hospital consistency. The trade-off is greater cybersecurity and operations complexity, which is justified by remote access and shared records. End-to-end testing must include offline operation, delayed/replayed messages, credential revocation, fail-over, recovery and data reconciliation.

Appendix – Execution evidence status

Compiler detected: gcc (Debian 14.2.0-19) 14.2.0
Platform detected: Linux 6.18.57 x86_64 GNU/Linux

Task06/file_copy.c

Compile: gcc -std=c11 -Wall -Wextra -O2 /mnt/data/CSA04-Assignment1/Task06/file_copy.c -o /mnt/data/CSA04-Assignment1/Task06/file_copy

Compile return code: 0

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Task07/process_control.c

Compile: gcc -std=c11 -Wall -Wextra -O2 /mnt/data/CSA04-Assignment1/Task07/process_control.c -o /mnt/data/CSA04-Assignment1/Task07/process_control

Compile return code: 0

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Task10/process_thread_bench.c

Compile: gcc -std=c11 -Wall -Wextra -O2 /mnt/data/CSA04-Assignment1/Task10/process_thread_bench.c -o /mnt/data/CSA04-Assignment1/Task10/process_thread_bench -pthread

Compile return code: 0

```
Created and reaped 1000 processes: 0.316079 seconds
Created and joined 1000 threads:  0.060130 seconds
```

Task05/system_calls.c

Compile: gcc -std=c11 -Wall -Wextra -O2 /mnt/data/CSA04-Assignment1/Task05/system_calls.c -o /mnt/data/CSA04-Assignment1/Task05/system_calls

Compile return code: 0

```
Smart hospital system-call demonstration.
```

References

• Silberschatz, Galvin and Gagne, Operating System Concepts, 10th Edition.

• Linux man-pages: read(2), write(2), fork(2), execve(2), waitpid(2), mmap(2), strace(1).

• Supplied CSA04 Operating Systems Assignment 1 brief.